공개 블록 I/O 트레이스 기반 Online Hot/Cold FTL 성능 분석

MSR Cambridge 및 Alibaba Cloud EBS workload 교차 검증

최종 기술 보고서

작성자: 이창엽

작성일: 2026 년 7 월

요약

본 프로젝트는 **실제 production block I/O trace 를 입력으로 받아 FTL 의 GC, WAF, valid-page migration 및 tail latency 를 분석하는 시뮬레이션 플랫폼**을 구축하고, workload 의 반복 갱신 특성을 이용한 Online Hot/Cold Placement 를 평가했다.

- Page-mapped FTL, Greedy GC, warm-up/measurement, dynamic SSD sizing 을 직접 구현했다.
- request-level analytical latency model 과 GC spillover attribution 으로 non-GC 요청의 queue-delay 전파를 분석했다.
- Alibaba 140.1 GiB 단일 압축 trace 를 전체 저장하지 않고 streaming 으로 첫 24 시간 525,559,379 개 요청을 분석했다.
- MSR 과 Alibaba 의 특성이 다른 4 개 workload 에서 동일 조건으로 Baseline 과 Proposed 를 비교했다.
- 130 개 단위 테스트로 mapping, GC, parser, streaming, classifier, attribution 및 결과 artifact 를 검증했다.

Workload	WAF 감소	Migration 감소	평균 지연 감소	p99 감소
MSR prxy_1	15.32%	26.31%	20.11%	21.38%
Alibaba dev4	34.03%	47.51%	45.42%	8.63%
Alibaba dev49	8.99%	17.17%	16.18%	0.78%
Alibaba dev740	-0.00%	-0.00%	-0.00%	-0.00%

핵심 결론: 정책 효과는 단순 write 비율보다 overwrite 정도와 persistent hot set 의 존재에 더 강하게 좌우되었다. 높은 overwrite workload 에서는 migration 과 WAF 가 크게 줄었고, hot set 이 없는 workload 에서는 Baseline 과 동일하게 퇴화했다.

목차

1. 문제 정의

2. 시스템 및 구현 구조

3. 데이터셋과 전처리

4. 제안 정책

5. 실험 설계

6. 실험 결과

7. 결과 해석

8. 구현 검증

9. 한계

10. 결론 및 향후 과제

부록 A. 재현 명령

부록 B. 면접 핵심 요약

1. 문제 정의

1.1 SSD GC 와 valid-page migration

NAND flash 는 기존 위치에 데이터를 덮어쓸 수 없으므로 update 는 새 page 에 기록되고 기존 page 는 invalid 상태가 된다. Free page 가 부족해지면 FTL 은 victim block 의 valid page 를 다른 위치로 이동한 뒤 block 을 erase 한다. 서로 다른 갱신 주기의 데이터가 같은 block 에 섞이면 GC 시 살아 있는 page 가 많아져 migration 비용이 커진다.

```
Mixed-lifetime placement
→ partially invalid block 증가
→ valid-page migration 증가
→ NAND program 증가
→ WAF 및 GC service time 증가
→ direct GC 와 spillover latency 증가
```

1.2 연구 질문

1. 최근 write 이력만으로 hot LPN 을 온라인 분류할 수 있는가?
2. Hot/Cold block 분리가 valid-page migration 과 WAF 를 줄이는가?
3. 감소한 GC 비용이 p95/p99 latency 와 GC spillover 감소로 이어지는가?
4. overwrite 가 낮거나 hot set 이 없는 workload 에서 정책은 어떻게 동작하는가?

2. 시스템 및 구현 구조

2.1 표준 입력 계약

```
timestamp,flow_id,op,lba,size

- timestamp: trace 기준 초
- flow_id: workload 또는 device 식별자
- op: R 또는 W
- lba: 시작 logical page number
- size: 4 KiB page 개수
```

Dataset adapter 와 FTL core 를 분리해 MSR, Alibaba 등 서로 다른 원본 schema 를 동일 simulator 에 입력할 수 있도록 구성했다.

2.2 주요 모듈

모듈	역할
baseline_ftl.py	Page mapping, host I/O 처리, GC accounting
metadata.py	LPN-PPN mapping, page/block state, erase count
gc_policy.py	Greedy victim selection
hot_cold_ftl.py	Online classifier 와 temperature-aware placement
trace_io.py	표준 trace parsing, working-set scan, SSD auto sizing

모듈	역할
latency_model.py	FCFS 단일 자원 analytical latency
latency_attribution.py	direct GC, spillover, workload queue 분류
workload_profile.py	overwrite, hotness, burst, sequentiality 분석
alibaba_streaming.py	S3 stream 기반 multi-device 추출
alibaba_workload.py	O(number of devices) pilot profiler

2.3 SSD geometry 와 capacity 모델

공개 trace 는 sparse LBA 를 포함하므로 최대 LBA 기준 SSD 를 생성하면 active working set 보다 지나치게 큰 장치가 된다. 따라서 distinct write working set 을 logical capacity 로 사용하고, 20% over-provisioning 과 chip/block alignment 를 적용했다.

```
page size              4 KiB
pages per block         64
chips                    4
target OP               20%
GC threshold             5%
reserved GC blocks       2
capacity basis           write working set
```

3. 데이터셋과 전처리

3.1 MSR Cambridge prxy_1

MSR enterprise trace 의 prxy_1 에서 100,000 개 요청을 표준 형식으로 변환해 사용했다. 34,450 개 write request 와 112,413 개 write page 를 포함하며 distinct written page 는 18,804 개였다.

3.2 Alibaba Block Traces 2020

Alibaba 원본은 약 140.1 GiB 의 단일 zstd 파일이며 1,000 개 virtual disk 의 요청이 interleaving 되어 있다. 전체 압축 해제본을 로컬에 저장하는 대신 다음 pipeline 을 구현했다.

```
AWS S3 stdout
→ zstd streaming decompression
→ O(device 수) profiler
→ 24 시간 candidate selection
→ multi-device 100k request extraction
→ local FTL simulation
```

- 첫 24 시간 분석 요청: 525,559,379 개
- 관찰 device: 818 개
- 로컬 처리 시간: 약 44 분 40 초
- 대표 역할: high-overwrite, mixed, low-overwrite burst

Workload	Write req	Write page	Overwrite page	Sequential	Max IOPS
MSR prxy_1	34.45%	35.21%	83.27%	3.76%	549
Alibaba dev4	96.51%	97.03%	87.65%	0.27%	2,636
Alibaba dev49	48.66%	31.78%	46.94%	2.44%	2,145
Alibaba dev740	100.00%	100.00%	14.45%	0.55%	3,050

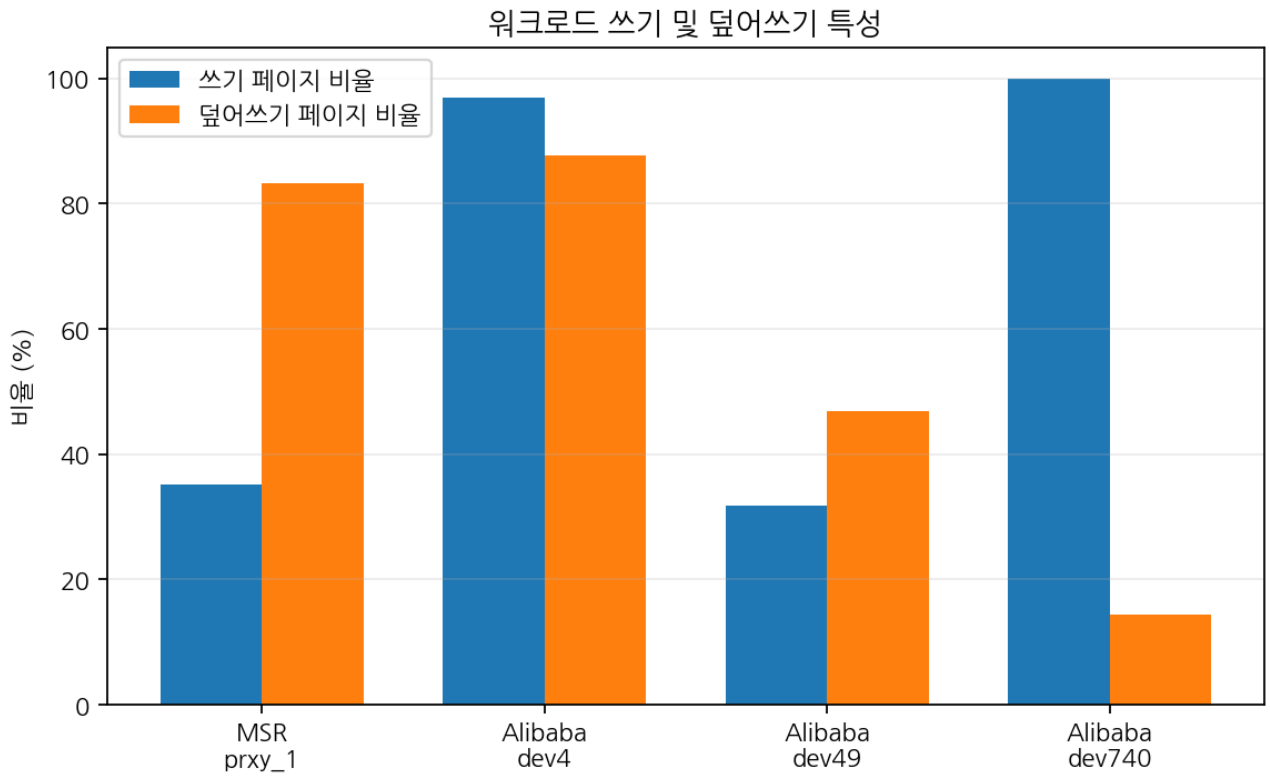


그림 1. 최종 workload 의 write page ratio 와 overwrite page ratio

4. 제안 정책: Online Hot/Cold Placement

4.1 Online hotness classifier

최근 host page write 이벤트만 유지하는 sliding window 를 사용한다. 현재 write 를 포함한 recent count 가 threshold 이상이면 해당 LPN 을 Hot 으로 승격하고, 오래된 이벤트가 window 밖으로 빠져 count 가 threshold 미만이 되면 Cold 로 강등한다. read 와 GC migration 은 classifier 를 갱신하지 않는다.

```
window_writes = 4096
hot_threshold = 8

first write                  → Cold
8th recent host write        → Hot promotion
recent count < 8 after aging → Cold demotion
```

4.2 Dynamic write frontier

- Hot 과 Cold 각각 active block 을 유지한다.
- 고정 capacity partition 대신 하나의 shared free-block pool 을 사용한다.
- GC victim policy 와 threshold 는 Baseline 과 동일하다.
- migration page 는 현재 classifier 결과를 따라 재배치하되 classifier count 를 증가시키지 않는다.
- 모든 최종 실험에서 fallback page 와 mixed-temperature block 은 0 이었다.

5. 실험 설계

5.1 가설

- H1: overwrite 와 hot-set 지속성이 높을수록 migration 과 WAF 감소폭이 크다.
- H2: migration/GC service 감소는 average/p95 latency 와 spillover 를 줄인다.
- H3: hot set 이 없으면 classifier 는 승격을 억제하고 Baseline 으로 퇴화한다.

5.2 통제 조건

항목	설정
Capacity	각 trace 의 write working set 기반, policy 간 동일
Over-provisioning	20% target
GC threshold	5%
Victim policy	Greedy
Warm-up / measurement	1 round / 1 round
Latency constants	command 20 us, read 80 us/page, program 200 us/page, erase 2000 us
Proposed parameter	threshold 8, window 4096

5.3 Analytical latency model

요청 도착 시간을 보존한 FCFS 단일 자원 queue 를 사용한다. 이 모델은 실제 SSD 의 절대 latency 를 예측하기 위한 것이 아니라 동일 모델에서 정책 간 service/queue 비용을 비교하고 GC 전파 원인을 분리하기 위한 것이다.

```
Read service = overhead + read_pages × read_page_us
Write service = overhead + host_program × program_page_us
              + migration × (read_page_us + program_page_us)
              + GC_count × erase_block_us
Latency = queue_delay + service_time
```

6. 실험 결과

6.1 Threshold 민감도

Threshold	WAF 개선	Migration 개선	p99 개선	Max 개선
2	7.22%	12.40%	8.84%	-17.64%
4	14.58%	25.05%	10.65%	9.62%
8	15.32%	26.31%	21.38%	14.74%

Threshold 2 에서는 max latency 가 17.64% 악화됐고, threshold 8 이 WAF, migration, p99, max latency 에서 가장 균형적인 결과를 보여 최종값으로 선택했다.

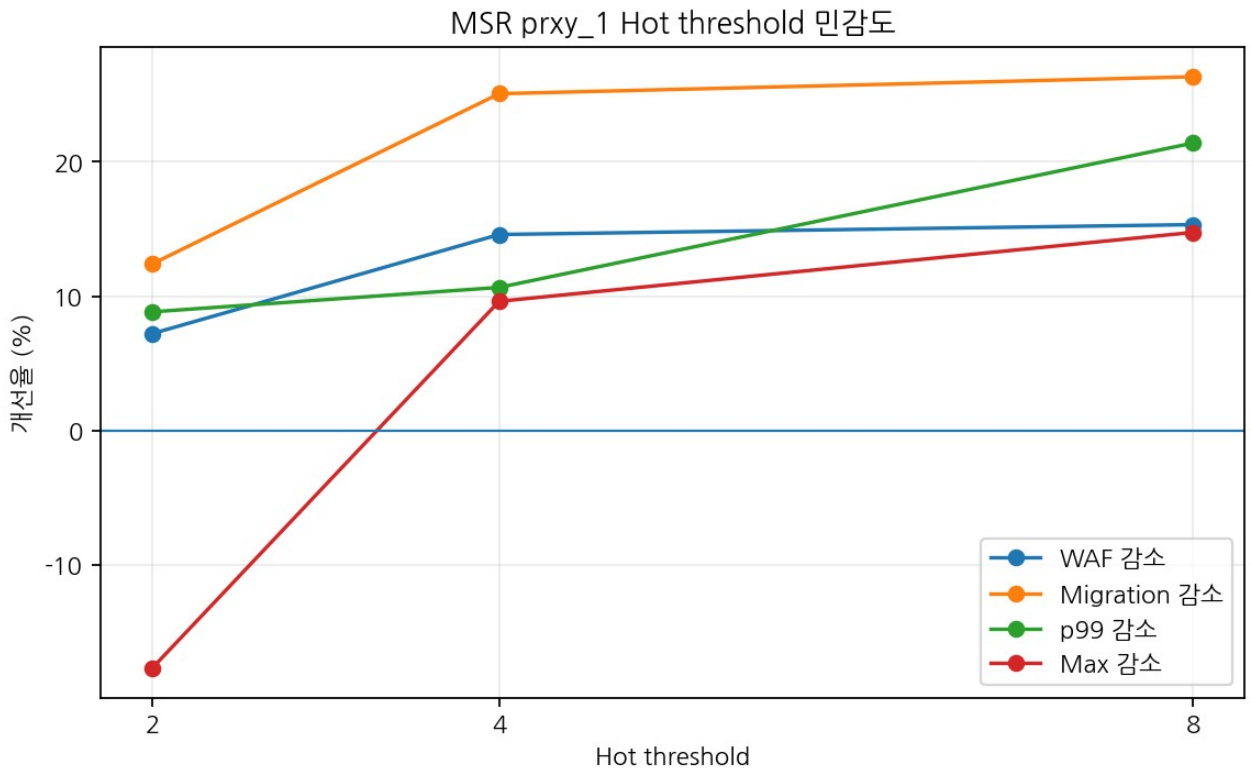


그림 2. MSR prxy_1 threshold 민감도

6.2 WAF 및 GC 비용

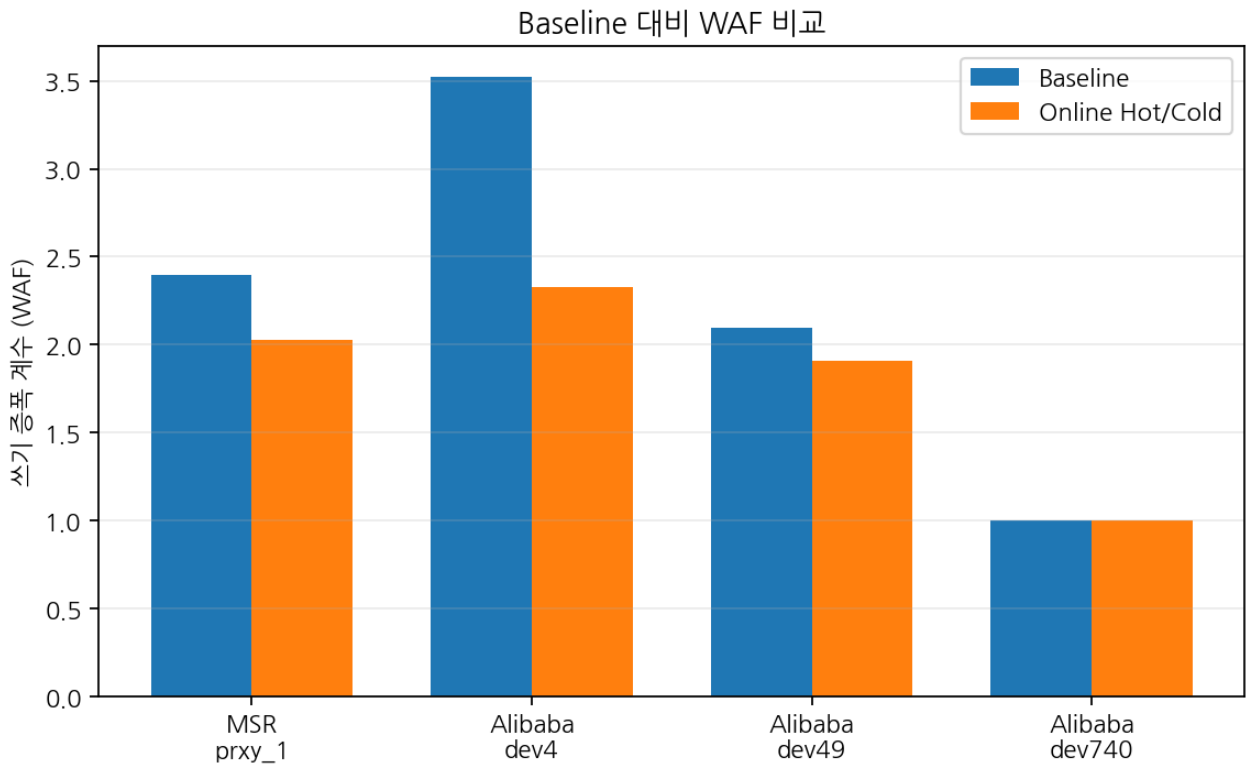


그림 3. Baseline 과 Online Hot/Cold 의 WAF

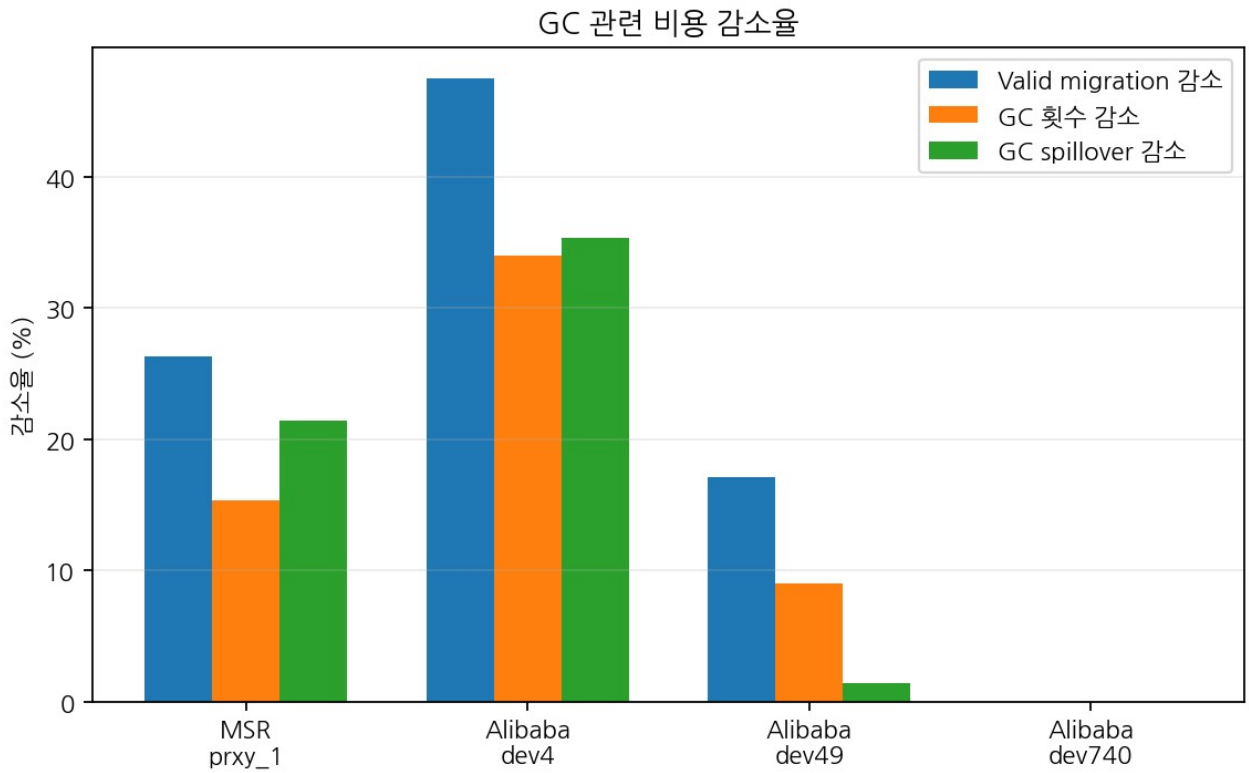


그림 4. valid migration, GC, spillover 감소율

Workload	Baseline WAF	Proposed WAF	WAF 감소	Migration 감소	GC 감소
MSR prxy_1	2.3936	2.0269	15.32%	26.31%	15.32%
Alibaba dev4	3.5242	2.3249	34.03%	47.51%	34.04%
Alibaba dev49	2.0992	1.9105	8.99%	17.17%	8.99%
Alibaba dev740	1.0000	1.0000	-0.00%	-0.00%	-0.00%

6.3 Latency 및 spillover

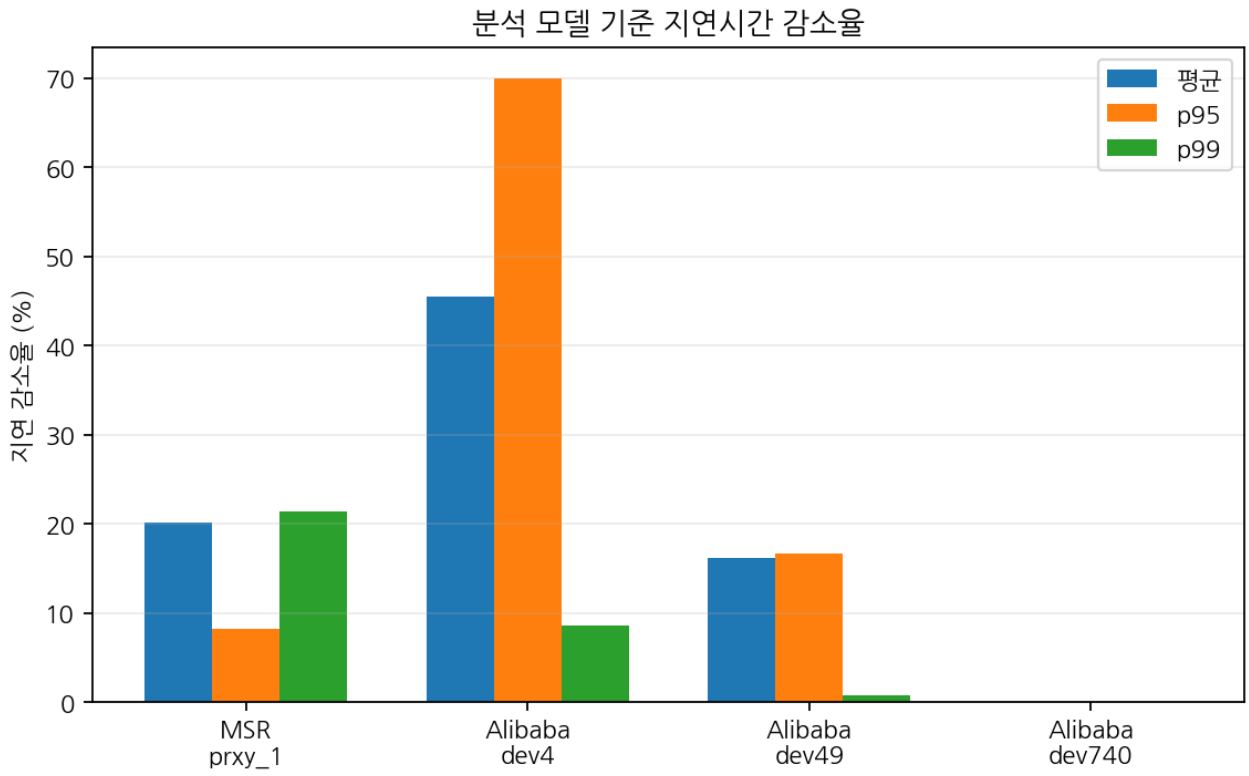


그림 5. Analytical latency 감소율

Workload	Avg 감소	p95 감소	p99 감소	p99.9 감소	Spillover 감소
MSR prxy_1	20.11%	8.22%	21.38%	13.43%	21.46%
Alibaba dev4	45.42%	69.98%	8.63%	4.37%	35.37%
Alibaba dev49	16.18%	16.60%	0.78%	2.53%	1.44%
Alibaba dev740	-0.00%	-0.00%	-0.00%	-0.00%	-0.00%

7. 결과 해석

7.1 Alibaba dev4: 강한 성공 사례

- Overwrite page ratio 87.65%, write page ratio 97.03%
- Hot host write 59,176 pages, Cold host write 55,485 pages
- 최종 Hot LPN 219 개, Hot block 72 개
- Migration 47.51%, WAF 34.03%, average latency 45.42%, spillover 35.37% 감소

반복 갱신 데이터를 Hot block 에 격리하면서 Cold block 의 유효 페이지 오염이 감소했고, GC 당 migration page 와 GC 횟수가 동시에 줄었다.

7.2 Alibaba dev49: 저장 효율 개선, 극단 tail 제한

- Overwrite page ratio 46.94%, mixed read/write workload
- WAF 8.99%, migration 17.17%, average latency 16.18% 감소
- p99 는 0.78%, spillover 는 1.44% 감소에 그침
- 최종 Hot LPN 0 개, 안정적인 hot set 이 작음

일부 반복 데이터를 분리해 평균 GC 비용은 감소했지만, burst 와 queue backlog 가 지배하는 극단 tail 은 공간 배치만으로 크게 해결되지 않았다.

7.3 Alibaba dev740: 안전한 퇴화

- Write 100%이지만 overwrite page ratio 는 14.45%
- Hot promotion 0 회, Hot write 0 page
- WAF 1.0, migration 0, 모든 성능 변화 0%

쓰기 비율이 높아도 재사용되는 주소가 적으면 Hot/Cold 분리의 대상이 없다. classifier 가 억지로 Hot 을 생성하지 않아 추가 NAND write 없이 Baseline 과 동일하게 동작했다.

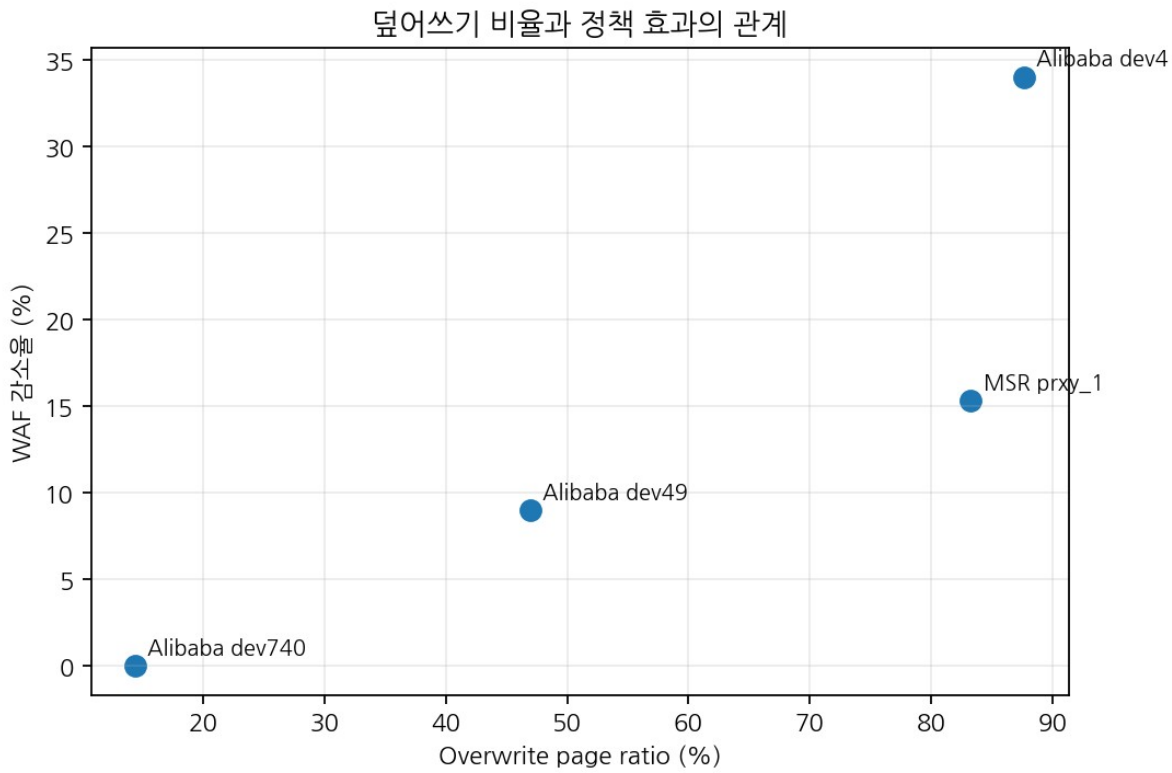


그림 6. Overwrite page ratio 와 WAF 감소율의 관찰 관계

8. 구현 검증

8.1 단위 테스트

프로젝트 전체 테스트를 재실행한 결과 130 개 테스트가 0.171 초에 통과했다. 주요 검증 범위는 다음과 같다.

- LPN-PPN mapping 과 overwrite invalidation
- free/valid/invalid page accounting
- Greedy victim selection 과 reserve handling
- warm-up/measurement counter delta
- online promotion/demotion 과 future-information 부재
- Alibaba schema parsing, streaming, checkpoint, multi-device extraction
- latency scope 와 direct/spillover attribution
- result CSV, JSON manifest, hash 와 sanity check 생성

8.2 Placement accounting

Workload	Fallback page	Mixed block	Promotion	Final hot LPN	Hot write share
MSR prxy_1	0	0	18	2	9.17%
Alibaba dev4	0	0	5571	219	51.61%

Workload	Fallback page	Mixed block	Promotion	Final hot LPN	Hot write share
Alibaba dev49	0	0	459	0	2.73%
Alibaba dev740	0	0	0	0	0.00%

모든 최종 실험에서 fallback 과 mixed block 이 0 이므로, 성능 차이는 공간 부족으로 분리 규칙을 우회한 결과가 아니다.

9. 한계

한계	영향
Latency 현실성	FCFS 단일 자원 모델로 channel/die 병렬성, NVMe queue, firmware CPU scheduling 을 반영하지 않는다.
Warm-up 구성	동일 100k trace 를 warm-up 과 measurement 에 반복해 독립 시간 구간 검증이 아니다.
Dataset 범위	MSR 1 개와 Alibaba 3 개 representative slice 로 전체 cloud population 을 대표하지 않는다.
Capacity 모델	write working set 기반 dynamic sizing 은 실제 provisioned device capacity 와 다르다.
Parameter 범위	threshold 2, 4, 8 만 비교했으며 window size sensitivity 는 제한적이다.
정책 독창성	Hot/Cold separation 개념 자체를 새 알고리즘으로 주장하지 않고, workload-driven online implementation 과 인과 검증에 의미를 둔다.

10. 결론 및 향후 과제

실제 enterprise 및 cloud block trace 에서 Online Hot/Cold Placement 를 평가한 결과, 높은 overwrite 와 지속적인 hot set 을 가진 workload 에서 valid-page migration 이 최대 47.51%, WAF 가 최대 34.03% 감소했다. 중간 overwrite 에서는 제한적인 개선을 보였으며, 낮은 overwrite 에서는 Hot 승격 없이 Baseline 과 동일하게 동작했다.

최종 인과관계: workload overwrite/hotness → online classification → block separation → migration 감소 → WAF/GC 감소 → spillover 및 latency 감소

향후 과제

- NAND channel/die parallelism 을 반영한 event-driven timing model
- 실제 SSD 또는 MQSim 결과를 이용한 latency calibration
- 시간상 독립된 warm-up/measurement 및 더 긴 trace slice

- workload phase 변화에 따른 adaptive threshold/window
- Hot/Cold placement 와 burst-aware GC scheduling 결합
- 더 많은 Alibaba device 에서 effect size 와 failure condition 통계화

부록 A. 재현 명령

A.1 테스트

```
python3 -m unittest discover -s tests -v
```

A.2 Workload profile

```
python3 -m simulator.run_profile_workload \
  --input results/alibaba_candidate_100k/device_4.standard.csv \
  --output-dir results/alibaba_profile_device_4 \
  --window-seconds 1.0 \
  --burst-percentile 95
```

A.3 Baseline vs Online Hot/Cold

```
python3 -m simulator.run_compare_hot_cold \
  --input results/alibaba_candidate_100k/device_4.standard.csv \
  --output-dir results/alibaba_compare_device_4_t8 \
  --warmup-rounds 1 \
  --measurement-rounds 1 \
  --hot-threshold 8 \
  --hot-window-writes 4096
```

부록 B. 면접 핵심 요약

- 문제는 GC 자체가 아니라 서로 다른 lifetime 의 page 혼합으로 발생하는 valid migration 이다.
- 정책은 trace 를 미리 보지 않고 최근 host write 만 사용한다.
- Alibaba 대용량 원본은 저장하지 않고 streaming profiler 와 extractor 로 해결했다.
- 가장 강한 결과뿐 아니라 효과가 제한적인 workload 와 0% workload 를 함께 제시했다.
- 절대 latency 가 아니라 동일 analytical model 에서 상대 변화와 원인 attribution 을 주장한다.